

9Lab

Reinforcement Learning

from MDP to RL and Q-Learning, Deep Q-Learning, and MARL

Mirko Viroli
`mirko.viroli@unibo.it`

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
ALMA MATER STUDIORUM—Università di Bologna, Cesena

a.a. 2025/2026

One slide sum-up on Reinforcement Learning

MDP and RL

- MDP is essentially DTMC plus actions/rewards, and the goal of maximising cumulative reward
- optimal policy can be found by Bellman equations, which badly scale in practice, though
- large state-space: addressable by Dynamic Programming, i.e. iterative algorithms
- unknown model or non-Markovianity: addressable by Monte Carlo approaches, i.e. simulation

Q-learning and beyond

- Q-learning: mixed DP and MC, with an off-policy learning that keeps improving the Q-table
- Deep Q-learning: represents Q-table by a DNN, with additional ingredients to properly converge
- MARL: many agents call for a wider model and specific algorithms

Starting point and goals

References

- 09-repo: from virtuale (<https://github.com/mviroli/asmd-public-models.git>)
- MARL and DQL repo:
<https://github.com/cric96/advanced-reinforcement-learning-asmd-code.git>

General goals

- play with basic RL and Q-learning
- play with Deep Q-Learning
- play with MARL

Operational Steps

Step 1 – Be operational with the RL repo and Q-learning examples

- Download the lab repository (<https://github.com/mviroli/asmd-public-models.git>)
- Import it in IntelliJ as an SBT project
- Get acquainted with Q-learning
 - ▶ Look at `scala/u09/examples`
 - ▶ Run `scala/u09/examples/TryQLearningMatrix` (it should produce the expected matrix with directions)
 - ▶ Check how variation of key parameters (ϵ , γ , α , episode length) affects learning
 - ▶ Check how learning gets more difficult as the grid size increases

Step 2 – Be operational with the DQN and MARL repo

- Download the DRL + MARL repository
(<https://github.com/cric96/advanced-reinforcement-learning-asmd-code.git>)
- Import it in IntelliJ as an SBT project
- Look at `scala/it/unibo/gym` to get acquainted with Deep Q-Learning
- Look at `scala/it/unibo/examples` to get acquainted with MARL

DESIGN BY Q-LEARNING

- Change the environment to support the notion of “corridor with obstacles” and make your robot learn to “zigzag walk” to avoid them
 - ▶ Update the QMatrix to also accept obstacles
 - ▶ Make it impossible to move in the direction of an obstacle, and give a negative reward if the robot tries to do it
 - ▶ Check if the agent learns to zigzag walk to avoid the obstacles
- By changing the environment, state, and rewards (adding holes, items, enemies, moving obstacles), you can make your “robot” learn virtually anything, e.g.:
 - ▶ “Program by learning” a robot to collect items one at a time and go back to the initial position
 - ▶ “Program by learning” a robot to move obstacles so as to hide from enemies

DESIGN BY MARL

- Update the row alignment example with an obstacle in the middle of the system (or more than one) and make the agents learn to coordinate to avoid it
- Start with a fixed obstacle and design it so that the reward is low when the agent approaches the obstacle
- Extend this example by changing the state space, e.g., by adding a vision range (8 cells) and checking for obstacles only within that range